

# Abstract Class vs Interface in Java - Cheat Sheet

---

Both let you define a contract that other classes implement. They look similar from a distance but solve different problems. Pick the wrong one and you'll be fighting the language for the next six months.

---

## Abstract Class

---

A class that can't be instantiated on its own. It holds state, constructors, regular methods, and abstract methods that subclasses must fill in.

```
public abstract class Vehicle {
    protected String name;
    protected int wheels;

    public Vehicle(String name, int wheels) {
        this.name = name;
        this.wheels = wheels;
    }

    public void start() {
        System.out.println(name + " starting...");
    }

    public abstract void honk(); // subclass must implement
}

public class Car extends Vehicle {
    public Car(String name) {
        super(name, 4);
    }

    @Override
    public void honk() {
        System.out.println("beep beep");
    }
}
```

Use when subclasses share state or partial logic.

---

## Interface

---

A pure contract. Lists method signatures and, since Java 8, can include `default` and `static` methods. No instance fields.

```

public interface Drivable {
    void accelerate();
    void brake();

    default void emergencyStop() {
        brake();
        System.out.println("emergency stop");
    }
}

public class Bike implements Drivable {
    @Override
    public void accelerate() {
        System.out.println("pedaling");
    }

    @Override
    public void brake() {
        System.out.println("squeezing brakes");
    }
}

```

Use to describe a capability that any class can plug into.

## Key Differences

| Feature                     | Abstract Class              | Interface                                             |
|-----------------------------|-----------------------------|-------------------------------------------------------|
| Keyword                     | <code>abstract class</code> | <code>interface</code>                                |
| Instantiation               | No                          | No                                                    |
| Constructor                 | Yes                         | No                                                    |
| Instance fields             | Yes                         | No (only public static final)                         |
| Method body                 | Yes (regular + abstract)    | default and static since Java 8, private since Java 9 |
| Inheritance limit           | One per class ( extends )   | Many per class ( implements )                         |
| Access modifiers on methods | Any                         | public by default (private allowed since Java 9)      |
| Multiple inheritance        | No                          | Yes                                                   |
| Typical use                 | “is-a” with shared state    | “can-do” capability                                   |

---

## When to Use Which

---

Pick **abstract class** when: - Subclasses share state or partial behavior. - You want to provide some method implementations and force subclasses to fill in the rest. - The relationship is genuinely “is-a” (Dog is an Animal, Car is a Vehicle). - You expect to add new methods later that need access to the shared state.

Pick **interface** when: - You’re describing a capability (Drivable, Serializable, Comparable). - A class needs to fit multiple roles (a Duck can be Swimmable, Flyable, and Quackable). - You’re defining an API contract for classes that have nothing else in common. - You want maximum flexibility for whoever implements it.

---

## Real-World Pairing

---

Java’s standard library often uses both together:

```
// Interface: the public contract
public interface List<E> { ... }

// Abstract class: shared skeleton for implementers
public abstract class AbstractList<E> implements List<E> { ... }

// Concrete class: extends the skeleton
public class ArrayList<E> extends AbstractList<E> { ... }
```

The interface tells the world what a `List` does. The abstract class saves implementers from rewriting the same boilerplate every time.

---

## Common Gotchas

---

- An interface field is implicitly `public static final`. You can’t have instance state, even by accident.
- A class can `extend` only one abstract class but can `implement` many interfaces.
- Adding a new method to an interface used to break every implementer. Java 8 `default` methods fixed that, but only when a sensible default exists.
- If a method is declared in both an extended abstract class and an implemented interface, the abstract class wins. Classes beat interfaces when the JVM resolves which method to call.
- If two interfaces define the same `default` method, the implementing class must override it to break the tie.
- `private` and `static` methods in interfaces help organize `default` method logic, but they’re not part of the contract.